

a great achievement for further high-level processing of a corpus. A word-frequency table can be programmed from the bow_corpus and dictionary, and convert the counter outcome to a DataFrame.

```
from collections import Counter
word_counts = Counter()
for doc in bow_corpus:
    for word_id, freq in doc:
        word = dictionary[word_id]
        word_counts[word] += freq
        print(word_id, freq, word, word_counts[word])

## 0 1 EWP 1
## 1 2 PHI 2
## 0 1 EWP 2
## 2 1 Elsevier 1
## 0 1 EWP 3
## 1 1 PHI 3
## 2 1 Elsevier 2
print('Type of word_counts: ', word_counts)
## Type of word_counts: Counter({'EWP': 3, 'PHI': 3, 'Elsevier': 2})
```

Using a Pandas DataFrame, a counter data type can easily be converted to a DataFrame for easy and elegant visualisation. This will give us a word-frequency table.

```
# Convert to a DataFrame for easier visualization
import pandas as pd
df = pd.DataFrame(word_counts.items(), columns=['Word',
```

```
'Frequency'])
df = df.sort_values(by='Frequency', ascending=False)
#Word frequency table
print(df)
##      Word  Frequency
## 0      EWP          3
## 1      PHI          3
## 2  Elsevier          2
```

Finally, matplotlib may be used to create a word-frequency plot of the corpus vocabulary in the manner described below:

```
# Plotting the frequency graph
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 6))
plt.bar(df['Word'], df['Frequency'], color='blue') # Show top 20 words
plt.title('Vocabulary vs. BoW Frequency', fontsize=14)
plt.xlabel('Vocabulary')
plt.ylabel('BoW Frequency')
plt.xticks(rotation=45, fontsize=14)
## ([0, 1, 2], [Text(0, 0, 'EWP'), Text(1, 0, 'PHI'), Text(2, 0, 'Elsevier')])
plt.show()
```

Complete Python code for the vocabulary study of a corpus consisting of four texts has been discussed here. Before the processing for vocabulary extraction, the entire corpus has been tokenised to clean with respect to

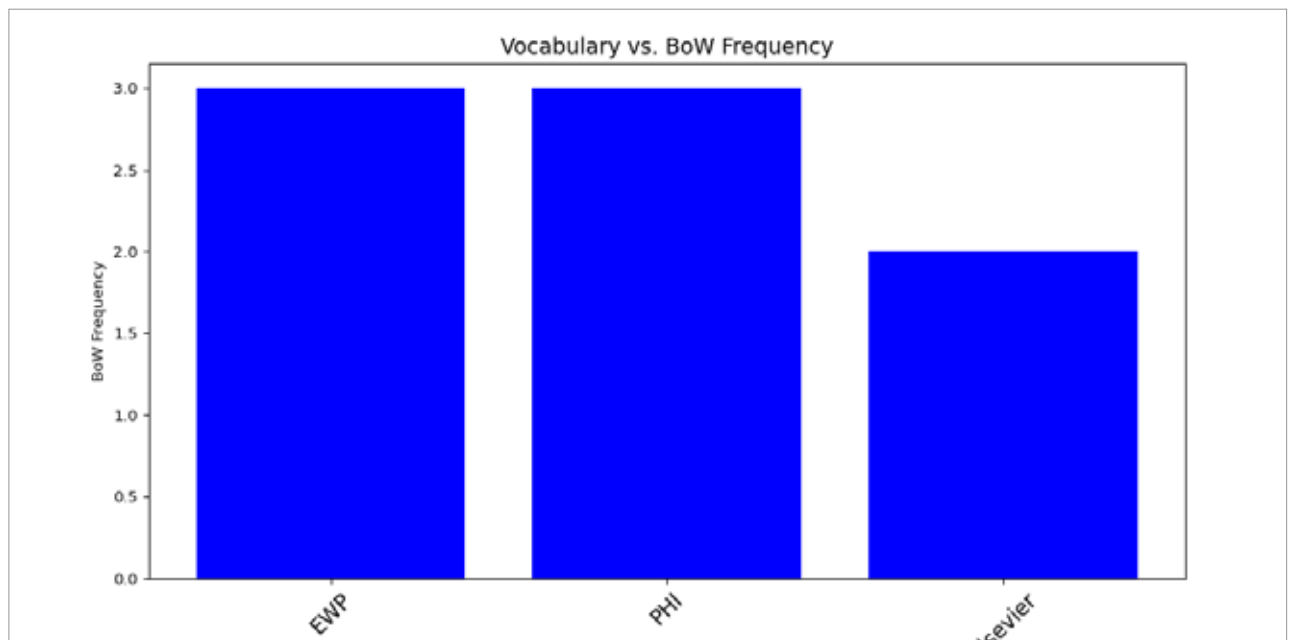


Figure 1: Word frequency plot of the given text